

NYC Crash Severity Prediction

Aansh Jha

```
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

RANDOM_STATE = 1234
DATA_FILE = Path("data") / "nyc_crashes_2024-06-30_to_2024-07-06.csv"
```

Question 1: Load and prepare the dataset

The outcome is whether anyone was injured. Predictors are limited to information describing the crash time, location, vehicle, and recorded contributing factor. Injury and fatality component counts are deliberately excluded because they directly reveal the outcome and would cause target leakage.

```
data = pd.read_csv(
    DATA_FILE,
    parse_dates=["crash_date"],
    dtype={"crash_time": "string", "zip_code": "string"},
)
data.columns = (
```

```

data.columns.str.strip()
.str.lower()
.str.replace(r"[^a-z0-9]+", "_", regex=True)
.str.replace(r"(?<=code)(\d)$", r"_\1", regex=True)
.str.strip("_")
)
data["crash_hour"] = pd.to_datetime(
    data["crash_time"], format="%H:%M", errors="coerce"
).dt.hour

numeric_features = ["crash_hour", "latitude", "longitude"]
categorical_features = [
    "borough",
    "contributing_factor_vehicle_1",
    "vehicle_type_code_1",
]
feature_columns = numeric_features + categorical_features

X = data[feature_columns].copy()
y = data["number_of_persons_injured"].gt(0).astype("int8")

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y,
)

numeric_pipeline = Pipeline(
    [
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

categorical_pipeline = Pipeline(
    [
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("one_hot", OneHotEncoder(handle_unknown="ignore")),
    ]
)

preprocessor = ColumnTransformer(
    [
        ("numeric", numeric_pipeline, numeric_features),
        ("categorical", categorical_pipeline, categorical_features),
    ]
)

```

```

    ]
)

print(f"Rows: {len(data):,}")
print(f"Training rows: {len(X_train):,}; test rows: {len(X_test):,}")
print(f"Injury rate: {y.mean():.1%}")

```

Rows: 1,676
 Training rows: 1,340; test rows: 336
 Injury rate: 44.9%

Question 2: Fit and evaluate a logistic model

```

logistic_model = Pipeline(
    [
        ("preprocess", preprocessor),
        (
            "model",
            LogisticRegression(max_iter=5000, random_state=RANDOM_STATE),
        ),
    ]
)
logistic_model.fit(X_train, y_train)

logistic_predictions = logistic_model.predict(X_test)
logistic_probabilities = logistic_model.predict_proba(X_test)[:, 1]
logistic_confusion = confusion_matrix(y_test, logistic_predictions)
tn, fp, fn, tp = logistic_confusion.ravel()

display(
    pd.DataFrame(
        logistic_confusion,
        index=["Actual non-severe", "Actual severe"],
        columns=["Predicted non-severe", "Predicted severe"],
    )
)
print(f"True positives: {tp}")
print(f"False positives: {fp}")
print(f"False negatives: {fn}")
print(f"True negatives: {tn}")
print(f"F1 score: {f1_score(y_test, logistic_predictions, zero_division=0):.3f}")

```

	Predicted non-severe	Predicted severe
Actual non-severe	145	40
Actual severe	92	59

True positives: 59
 False positives: 40
 False negatives: 92
 True negatives: 145
 F1 score: 0.472

Question 3: Fit an L1-regularized logistic model

```

l1_model = Pipeline(
    [
        ("preprocess", preprocessor),
        (
            "model",
            LogisticRegressionCV(
                Cs=10,
                cv=5,
                penalty="l1",
                solver="liblinear",
                scoring="f1",
                max_iter=5000,
                random_state=RANDOM_STATE,
            ),
        ),
    ],
)
l1_model.fit(X_train, y_train)

l1_predictions = l1_model.predict(X_test)
l1_probabilities = l1_model.predict_proba(X_test)[: , 1]
best_c = float(l1_model.named_steps["model"].C_[0])
print(f"Best inverse regularization strength (C): {best_c:.6g}")

```

Best inverse regularization strength (C): 21.5443

Question 4: Compare model performance

```

def classification_metrics(y_true, predictions, probabilities):
    return {
        "Accuracy": accuracy_score(y_true, predictions),
    }

```

```

        "Precision": precision_score(y_true, predictions, zero_division=0),
        "Recall": recall_score(y_true, predictions, zero_division=0),
        "F1": f1_score(y_true, predictions, zero_division=0),
        "ROC AUC": roc_auc_score(y_true, probabilities),
    }

comparison = pd.DataFrame(
    {
        "Logistic": classification_metrics(
            y_test, logistic_predictions, logistic_probabilities
        ),
        "L1 regularized": classification_metrics(
            y_test, l1_predictions, l1_probabilities
        ),
    }
)
display(comparison.round(3))

```

	Logistic	L1 regularized
Accuracy	0.607	0.595
Precision	0.596	0.577
Recall	0.391	0.371
F1	0.472	0.452
ROC AUC	0.618	0.610

These test-set results are a realistic assessment of association, not evidence of causality. They are also expected to be lower than the earlier near-perfect scores because the corrected models no longer use injury and fatality counts to predict whether an injury occurred.